

سند استاندارد معماری رویدادها و راهبرد ایونت باس پلتفرم

Event Architecture & Event Bus Strategy Standard

نسخه: ۱٫۰ (Freeze)

وضعیت - Architectural Decision Document (ADD): مصوب و نهایی

مرجع فنی: اکوسیستم ۱۱.x (Laravel) و ۱۶+ PostgreSQL مدل مونوپلیت ماژولار ()

=====

=====

۱. هدف سند

این سند استانداردهای حاکم بر طراحی، انتشار، مدیریت و مصرف رویدادها (Events) را در پلتفرم چندمستأجری HamarehERP تعریف می‌کند.

هدف اصلی این راهبرد :

- ایجاد مکانیزم ارتباطی ناهمگام (Asynchronous) و سست‌پیوسته (Loose Coupling) بین ماژول‌های مستقل سیستم .
- جلوگیری از وابستگی مستقیم، ارجاع‌های چرخشی یا کوپلینگ سخت لایه کد .
- تضمین پایداری تراکنش‌ها و یکپارچگی نهایی داده‌ها (Eventual Consistency) در محیط مونوپلیت ماژولار .
- آماده‌سازی هم‌زمان زیرساخت لایه کد برای مهاجرت بدون دردسر به ساختار میکروسرویس (Microservices Ready) در فازهای بعدی پروژه .

=====

=====

۲. راهبرد زیرساختی فاز اول (In-Backend Outbox Queue)

با توجه به متدولوژی Lean و معماری مونوپلیت ماژولار در فاز اول پروژه، پلتفرم به جای درگیر شدن زود هنگام با کارگزاران خارجی و پیچیده پیام) مانند Kafka یا RabbitMQ ، از زیرساخت پیش فرض و بسیار پایدار لاراول (Laravel Queue) استفاده خواهد کرد .

- این فرآیند از طریق مدیریت صف در پایگاه داده (Database Queue Driver) و مانیتورینگ دقیق پس‌زمینه انجام می‌شود .

- پلتفرم کدهای ماژول‌ها را به گونه‌ای ایزوله می‌کند که تغییر واسط پیام از لاراول به یک بروکر خارجی در آینده، صرفاً با تعویض کلاس درایور در لایه کانفیگ پلتفرم انجام شود و نیازی به تغییر بیزینس لاجیک درون ماژول‌ها نباشد.

=====

=====

۳. چالش تراکنش‌های مالی و پترن حیاتی Transactional Outbox

بزرگترین ریسک سیستم‌های رویدادمحور، عدم همگام‌سازی تراکنش دیتابیس با وضعیت انتشار رویداد است. اگر داده در دیتابیس یک ماژول (مثلاً ماژول فروش) ذخیره شود اما به دلیل خطای شبکه یا کراش سرور، رویداد آن صادر نشود، سیستم دچار تضاد داده‌ای (Data Inconsistency) شدید می‌شود.

- قانون غیرقابل تغییر: انتشار مستقیم و آنی رویدادهای همگام (In-Memory Synchronization) یا سپردن رویدادها به حافظه موقت بدون ثبت دیتابیس ممنوع است.
- الزام پترن Outbox: تمامی ماژول‌ها مکلفند رویدادهای خود را در همان تراکنش واحد پایگاه‌داده (Database Transaction) ماژول صادرکننده، ابتدا درون جدول محلی لاراول (jobs یا outbox_events) کامیت و ذخیره کنند.
- واگذاری ناهمگام: دیمن‌های پس‌زمینه (Laravel Queue Workers) از طریق دستور php artisan queue:work وظیفه دارند به صورت ناهمگام، این رویدادها را از جدول اوتباکس دیتابیس خوانده، بازسازی کرده و به لیسنرهای ماژول‌های مقصد تحویل دهند. اگر فرآیند با شکست مواجه شود، تراکنش اصلی در دیتابیس حفظ شده و رویداد مجدداً تلاش (Retry) خواهد شد.

=====

=====

۴. اصول ساختار داده‌ای رویدادها (Event Payload Standard)

تمامی کلاس‌های رویداد صادر شده در پلتفرم باید از یک ساختار استاندارد، مستقل و یکپارچه تبعیت کنند. هر رویداد یک شیء داده‌ای خالص (DTO) شامل ویژگی‌های زیر است:

۴.۱. شناسه‌های فنی و بافتاری:

- event_id: یک شناسه منحصر به فرد جهانی (UUID) برای هر نمونه رویداد.

- event_type: نام رسمی و بیزینسی رویداد بر اساس زبان مشترک پلتفرم (مانند InvoiceCreated).
 - tenant_id: شناسه دقیق مستأجر صاحب رویداد (UUID) جهت حفظ ۱۰۰٪ ایزوله‌سازی داده‌ها .
 - timestamp: زمان دقیق وقوع رویداد به فرمت استاندارد ISO ۸۶۰۱ همراه با منطقه زمانی (TIMESTAMP TZ).
 - triggered_by: شناسه کاربر عامل و صادرکننده رویداد .
- ۴.۲. بدنه اطلاعات بیزینسی: (Payload)
- بدنه رویداد نباید حاوی نمونه کلاس‌ها یا اشیاء زنده دیتابیس (مانند مدل Eloquent) باشد .
 - بدنه صرفاً باید شامل داده‌های متنی خالص، شناسه‌های منطقی (Logical UUIDs) و مقادیر عددی ثابت باشد تا در زمان تبدیل به متن (Serialization) به فرمت (JSON) و ذخیره در صف، اطلاعات مخدوش نشوند .

=====

=====

۵. قوانین صادرکنندگان رویداد (Event Publishers)
- ماژول صادرکننده مالک مطلق چرخه حیات داده خود است .
- ماژول صادرکننده پس از انجام موفق عملیات داخلی و بسته‌شدن تراکنش محلی، رویداد را به پلتفرم اوتباکس تحویل می‌دهد .
 - صادرکننده هیچ اطلاعی از اینکه چه ماژول‌هایی گوش به زنگ این رویداد هستند (Loose Coupling) ندارد .
 - نام‌گذاری رویدادها باید کاملاً با فعل گذشته انگلیسی و منطبق بر اصطلاحات بیزینسی انجام شود (مثلاً، UserRegistered , InventoryReserved) :

=====

=====

۶. قوانین مصرف‌کنندگان رویداد (Event Consumers / Handlers)
- ماژول‌های دریافت‌کننده پیام مکلف به رعایت اصول زیر هستند :

۶.۱. ضمانت یکبارپذیری (Idempotency Requirement)

به دلیل ماهیت صف‌های ناهمگام و پترن Outbox، این احتمال فنی وجود دارد که یک رویداد به دلیل خطاهای شبکه یا تایم‌اوت‌ها، بیش از یک بار به ماژول مقصد ارسال شود (At-least-once delivery).

- تمامی کدهای EventHandler مکلفند پیش از پردازش منطق رویداد، شناسه event_id را در یک جدول اختصاصی به نام processed_events چک کنند. اگر این آیدی قبلاً پردازش شده بود، رویداد باید فوراً بدون اعمال مجدد و بدون تولید خطا، نادیده گرفته (Skip) شود.

۶.۲. بازسازی فوری: Tenant Context

لیسنرهای صف در ثانیه اول پردازش هر پیام، باید متغیر tenant_id موجود در Payload رویداد را استخراج کرده و ساختار امنیتی مستأجر را در کانتینر مرکزی لاراول ست کنند (SET local (app.current_tenant_id) تا کدهای داخلی ماژول تحت فیلترهای امنیتی صحیح اجرا شوند.

=====

====

۷. استراتژی خطا و جدول مرگ رویدادها (Dead Letter Queue - DLQ)

اگر پردازش یک رویداد در ماژول مقصد به دلیل باگ‌های نرم‌افزاری یا خطاهای سیستمی با شکست مواجه شود، نباید کل ترد (Thread) یا صف سیستم قفل شود.

- پلتفرم لاراول باید برای حداکثر تعداد تلاش مجدد (Max Retries = ۳) با فاصله زمانی پلکانی (Exponential Backoff) پیگیری شود.

- در صورت شکست نهایی پس از ۳ بار تلاش، رویداد باید به صورت خودکار به جدول خطای مرکزی پلتفرم یا همان لایه داخلی (Dead Letter Queue) جدول failed_jobs (لاراول) منتقل شود.

- یک سیستم مانیتورینگ اختصاصی در لایه SaaS Admin پلتفرم باید این خطاها را به ادمین‌های فنی نمایش دهد تا امکان بررسی، اصلاح کدهای ماژول و ارسال مجدد دستی (Retry) پیام فراهم باشد.

=====

====

۸. قانون نهایی

سیستم مدیریت رویدادها و راهبرد ناهمگام اوتباکس پلتفرم، ضامن یکپارچگی اطلاعات مالی، انبارداری و زنجیره تامین سیستم HamarehERP است. هرگونه تلاش برای دور زدن این ساختار و ایجاد اتصالات همگام یا تغییر مستقیم داده‌ها در دیتابیس ماژول‌های دیگر، خط قرمز پروژه محسوب شده و کدهای مربوطه مردود خواهند شد .